5. Linux File System Hierarchy and File Permissions

File system hierarchy

Everything in Linux starts from `/` (root). There are no drive letters like C: or D:.

Directory	What it holds	Why it matters in security
`/etc`	System and application configuration files (`passwd`, `shadow`, `ssh/`)	Attackers target it for credentials and config changes
`/var/log`	Log files (`auth.log`, `syslog`, `apache2/`)	First place a SOC analyst checks for login attempts and errors
`/home`	Personal folders of normal users	User data, SSH keys, shell history
`/tmp`	Temporary files, cleared on reboot, writable by everyone	Malware often drops files here because anyone can write

Others worth knowing: `/bin` (basic commands), `/root` (root user's home), `/usr` (installed programs), `/dev` (devices), `/proc` (running processes info).

File permissions

Run `ls -l` and you get something like:

```
-rwxr-xr-- 1 harini devs 1200 Oct 20 10:00 script.sh
```

- First character: file type (`-` file, `d` directory, `l` link)
- Next nine: permissions for **owner**, **group**, **others**, three each
- `r` = read (4), `w` = write (2), `x` = execute (1)

So `rwxr-xr--` is 754: owner can do everything, group can read and execute, others can only read.

chmod (change permissions)

```
chmod 755 script.sh      # owner rwx, group r-x, others r-x
chmod u+x script.sh      # add execute for owner
chmod go-w file.txt      # remove write from group and others
chmod -R 750 project/    # apply recursively to a folder
```

chown (change owner)

```
sudo chown harini file.txt             # change owner
sudo chown harini:devs file.txt        # change owner and group
sudo chown -R harini:devs /srv/share   # recursive
```

SUID (Set User ID)

When a file has the SUID bit, it runs with the permissions of the **file owner**, not the person running it. It shows as an `s` in the owner's execute spot.

```
ls -l /usr/bin/passwd
-rwsr-xr-x 1 root root 59976 /usr/bin/passwd
```

This is how a normal user can change their own password even though `/etc/shadow` is only writable by root.

```
chmod u+s program        # set SUID
chmod 4755 program       # same thing in numeric form
find / -perm -4000 2>/dev/null     # find all SUID files
```

Why it matters: if a SUID file is owned by root and has a flaw (or is something like `find` or `vim` that can spawn a shell), an attacker can use it for privilege escalation. Finding SUID files is a standard step in Linux privilege escalation and in hardening checks.